

# GarudaESE / EV60Y51A — Firmware Port Reference

6-step sensorless BLDC ESC firmware for the **EV60Y51A V1.0** board. This document describes the firmware features, the verified pin map, the current-sense / BEMF architecture, the configurable options, the protection layer, and the known caveats. Read this together with `GarudaESE_BringUp_Plan.md` before powering a motor.

Status: **builds clean** (MPLAB XC-DSC v3.30, dsPIC33AK-MC\_DFP 1.4.172) and flashes. Verified at the register-encoding / compile / flash level and against the board schematic + the two parent codebases. **NOT yet validated on a spinning motor** — see the bring-up plan.

## 1. Hardware

| Item             | Part / value                                                                                                                                                                         |
|------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| MCU              | <b>dsPIC33AK256MC506-E/M7</b> , 64-VQFN, Fcy = 200 MHz                                                                                                                               |
| Gate driver      | <b>ATA6847T-5033LH</b> — SPI-configured 3-phase driver, internal HS charge pump, internal dead-time, 3 op-amps + 3 BEMF comparators                                                  |
| MOSFETs          | <b>BSC028N06NS</b> (60 V, ~2.8 mΩ)                                                                                                                                                   |
| Shunts           | <b>4 × 2 mΩ</b> ISA-WELD (R46/47/48 = U/V/W low-side legs, R57 = DC bus)                                                                                                             |
| Bus              | <b>25 V / 6S — HARD LIMIT for this campaign</b>                                                                                                                                      |
| Comms            | GSP over the DEBUG UART (UART1, RD7/RD8); CAN (ATA6561) on RC10/RC11                                                                                                                 |
| Source schematic | ESC_Board_Release/ (Altium) — MCU I0.SchDoc , BRIDGE , SENSING.SchDoc , POWER SUPPLY , GATE DRIVER.SchDoc ; rendered EV60Y51A-V1.0.pdf (pg3 power/gate, pg4 MCU, pg5 bridge/sensing) |

## 2. Provenance

Assembled from two Project Garuda code bases: - **Control engine + GSP** from the AK512 `dspic33AKESC` tree (6-step `bemf_zc` software ZC, GSP v3 protocol, state machine, startup, PI). - **ATA6847 + SPI driver** from `garuda-ak-ata-esc` (retargeted SPI1 → SPI2). - A **new board layer** (`hal/port_config.*`, `hal/garuda_board.h`, `hal/hal_adc.*`) for the EV60Y51A pin map and the 5-core ADC.

FOC code (`foc/`) is present but **dormant** (all `FEATURE_FOC*` = 0); this is a clean 6-step build.

## 3. Build

```
cd garuda_ese.X && make CONF=default      # XC-DSC v3.30 on PATH
```

or open `garuda_ese.X` in MPLAB X and Build. The root `Makefile` + `nbproject/Makefile-*.mk` are committed; machine-specific generated files (`Makefile-local-*.mk`, `Makefile-genesis.properties`, `private/`) are gitignored. Build footprint: program ≈ 26 %, data ≈ 10 %.

## 4. Verified pin map

### 4.1 PWM (motor outputs)

PG1 = U, PG2 = V, PG3 = W. High-side is **active-LOW** at the ATA inputs ( P0LH = 1 ), low-side active-high ( P0LL = 0 ). MCU dead-time per-profile (~300 ns) **plus** the ATA's internal dead-time. Switching = **45 kHz**.

| Generator | H pin | L pin |
|-----------|-------|-------|
| PG1 (U)   | RD2   | RD3   |
| PG2 (V)   | RD0   | RD1   |
| PG3 (W)   | RC3   | RC4   |

### 4.2 Analog / ADC (schematic-verified, every channel)

| Signal                     | dsPIC pin | Amplifier    | ADC core.chan | PINSEL (ANx) |
|----------------------------|-----------|--------------|---------------|--------------|
| Iu                         | RA2       | dsPIC OA1    | AD1 CH0       | 0 (AD1AN0)   |
| Iv                         | RB0       | dsPIC OA2    | AD2 CH0       | 0 (AD2AN0)   |
| Iw                         | RA5       | ATA6847 CSA2 | AD3 CH3       | 0 (AD3AN0)   |
| Ibus                       | RB4       | ATA6847 CSA3 | AD4 CH0       | 0 (AD4AN0)   |
| BEMF_U (divider)           | RB5       | —            | AD3 CH0       | 1 (AD3AN1)   |
| BEMF_V (divider)           | RB8       | —            | AD2 CH1       | 4 (AD2AN4)   |
| BEMF_W (divider)           | RA1       | —            | AD5 CH0       | 1 (AD5AN1)   |
| BEMF_N (virtual neutral)   | RA11      | —            | AD5 CH1       | 2 (AD5AN2)   |
| VBUS (divider 0.0769)      | RA7       | —            | AD5 CH2       | 0 (AD5AN0)   |
| TEMP (NTC)                 | RA0       | —            | AD3 CH1       | 5 (AD3AN5)   |
| Speed/POT (test point TP5) | RA6       | —            | AD3 CH2       | 2 (AD3AN2)   |

**Speed/POT throttle:** a pot wired to test point **TP5 = RA6 = AD3AN2** is the boot throttle source when `FEATURE_ADC_POT=1` (turn the knob). GSP stays available at runtime via `SET_THROTTLE_SRC`. The dsPIC OA3 is unused so RA6 (OA3IN-) is free for this analog input. An unpopulated/floating TP5 reads non-zero → the board won't arm (arm requires throttle=0), which is safe.

**Cross-checked against the authoritative dsPIC33AK256MC506.xlsx pin map (2026-06-22): all functional ADC channels (Iu/Iv/Iw/Ibus, BEMF\_U/V/W/N, VBUS, TEMP) + the PWM pins, SPI2 pins, debug-UART, CAN, ATA\_BEMF GPIOs, LED and the RD5 arm/DShot pin MATCH.** Two notes: (1) the xlsx lists Speed/POT on RA8/AD5AN3, but the **schematic** (TP5 wire) puts it on **RA6/AD3AN2** — the xlsx is wrong here (RA8 is the UREF net); the firmware follows the schematic. (2) SPI2 pins match; the data-line *direction* (RB11=out, RC9=in) follows the board net routing, which the function-only xlsx does not capture. | ATA\_BEMF\_U (digital ZC) | RA9 | ATA cmp | GPIO | ANSEL=0 | | ATA\_BEMF\_V (digital ZC) | RA10 | ATA cmp | GPIO | ANSEL=0 | | ATA\_BEMF\_W (digital ZC) | RB9 | ATA cmp | GPIO | ANSEL=0 |

**Key insight:** the dsPIC OA3 is **not used** (W is amplified by the ATA), so OA3's input pins are reused: **RB5 (AD3AN1) = BEMF\_U, RA6 (AD3AN2) = Speed**. The AD3 core therefore carries Iw + BEMF\_U +

TEMP + Speed.

The three BEMF phase dividers are on **separate ADC cores** (AD3/AD2/AD5), sampled **simultaneously** every PG1TRIGA — there is **no Phase-A/C mux** and no settle penalty (unlike the MCLV base this was ported from). ADC interrupt fires on `_AD1CH0Interrupt` (lu completion) — the ISR anchor.

#### 4.3 SPI2 ↔ ATA6847 (direction follows the board net routing, not the pin names)

| Net      | dsPIC                                | ATA pin                 |
|----------|--------------------------------------|-------------------------|
| SPI_SCK  | RB10 (RP27) → SCK2OUT                | SCK (47)                |
| SPI_SDI  | <b>RB11 (RP28) → SDO2 (data OUT)</b> | SDI (46, data into ATA) |
| SPI_SDO  | <b>RC9 (RP42) → SDI2 (data IN)</b>   | SDO (45, data from ATA) |
| SPI_NCS  | RC8 (GPIO, idle high)                | NCS (44)                |
| SPI_NIRQ | RC6 (GPIO in)                        | NIRQ (43)               |

**Caveat (caught the hard way):** the board author named the nets after the dsPIC pins' *default* functions (RB11=SDI2, RC9=SDO2) but wired SDI ↔ SDI / SDO ↔ SDO — backwards for SPI. PPS lets any RPn be SDO2-out or SDI2-in, so the firmware follows the **routing** (RB11=output, RC9=input), not the names. Mode: CKP=0, CKE=0, SMP=1, 16-bit, ENHBUF (CKP=1 is known to break ATA reads).

#### 4.4 J1 signal connector (BM12B-GHS-TBT, JST GH 12-pin)

| Net            | dsPIC pin  | Use                                             |
|----------------|------------|-------------------------------------------------|
| +3.3 V / GND   | —          | power/ground                                    |
| MCLR           | MCLR       | reset                                           |
| PGD / PGC      | RB3 / RB4* | programming (ICSP)                              |
| DShot          | RD5        | DShot in / legacy momentary Start-Stop          |
| TELE_TX        | RD6        | <b>free</b> (telemetry UART unused)             |
| <b>TELE_RX</b> | <b>RD4</b> | <b>dedicated ARM toggle switch</b> (active-low) |
| DEBUG_TX       | RD7        | <b>GSP</b> UART1 TX                             |
| DEBUG_RX       | RD8        | <b>GSP</b> UART1 RX                             |

\*PGC shares RB4 with the bus-current ADC in normal operation; it is the programming clock only during ICSP.

#### 4.5 Other

LED = RC0; GSP UART1 TX=RD7/RX=RD8; ARM toggle = RD4 (J1), legacy momentary = RD5; CAN TX=RC10/RX=RC11, STBY=RB7; OSC = internal 8 MHz FRC → PLL (an 8 MHz crystal is fitted but unused — same PLL input either way).

## 5. Current sense (mixed amplification)

---

The EV60Y51A uses **both** amplifier resources: - **I<sub>u</sub>, I<sub>v</sub>** → the **dsPIC internal op-amps** OA1/OA2 (external-resistor gain,  $\sim \times 36$ ). - **I<sub>w</sub>, I<sub>bus</sub>** → the **ATA6847 op-amps** CSA2/CSA3 (internal gain  $\approx 16$ ), whose outputs drive plain dsPIC ADC inputs (AD3AN0, AD4AN0).

Per the ATA6847 datasheet the BEMF comparators conflict **only with CSA1**; CSA2/CSA3 run alongside BEMF. The firmware therefore enables `CSA2EN|CSA3EN` (not CSA1) and keeps the BEMF path free.

**Consequence for calibration:** I<sub>u</sub>/I<sub>v</sub> and I<sub>w</sub>/I<sub>bus</sub> read on **different counts/amp** scales (different gain + the same 2 mΩ shunt). Any amp-based threshold must be calibrated per channel — see §9.

## 6. PWM / commutation

---

- Center-aligned complementary, PG1 master / PG2-PG3 slaves (slaved SOC update).
- `PGxIOCON` is split on this device into **IOCON1** (PENH/PENL/POLH/POLL/PMOD, set once at init) and **IOCON2** (override/fault data, written per commutation). The 6-step override words target IOCON2: `OVRENH=bit21`, `OVRENL=bit20`, `OVRDAT=bits[13:12]`. `POLH=1` inverts only the final H pin (after the override mux), so the override encoding is identical to a non-inverted board.
- Override words: `PWM_ACTIVE=0x0` (complementary), `LOW=0x00301000` (H off/L on), `FLOAT=0x00300000` (both off). Duty written slaves-before-master.
- Dead-time is **kept** (per-profile MCU DT alongside the ATA internal DT); `MIN_DUTY = 2 × DEADTIME_COUNTS` depends on it — do **not** zero it.
- ADC trigger = PG1TRIGA, sample at the ON-pulse centre (the duty-proportional ZC model expects this).

## 7. ATA6847 bring-up

---

`HAL_InitPeripherals()` order: SPI2 → ATA init → OA1/OA2 → ADC → PWM (overridden LOW) → Timer1 → ClearFaults → **EnterGduNormal() (checked)**.

- ATA init configures WD, ILIM, SCPCR, CSCR (op-amp gain), GDU CR1-4 (incl. `BEMFEN=1`), DOPMCR (operating mode, verified + retried), SIECER masks.
- `EnterGduNormal()` enables CSA2|CSA3, transitions GDU OFF → STANDBY → NORMAL, polls `DSR1.GDUS` (rejects 0xFF false-success). Its result sets `g_ataReady`.
- **Gates cannot switch until** the firmware sets DOPM=Normal + GDU Normal; the PWM `OVRENx` overrides hold all outputs LOW until the state machine enters ALIGN.

## 8. Zero-crossing detection

---

Software ZC ( `motor/bemf_zc.c`, `FEATURE_BEMF_CLOSED_LOOP=1` ) reads the floating phase's divider ADC and compares against a **neutral/threshold** with deadband, per-phase gain/offset, blanking, and noise filtering; commutation is scheduled 30° (minus timing advance) after a confirmed ZC.

## 8.1 Selectable neutral model — `ZC_NEUTRAL_MODEL` (`garuda_config.h`)

| Value                                     | Model               | Notes                                                                                       |
|-------------------------------------------|---------------------|---------------------------------------------------------------------------------------------|
| <code>ZC_NEUTRAL_DUTY</code><br>(default) | Vbus·duty / divisor | ON-center model; tracks the float's PWM-ON level. Bench-proven.                             |
| <code>ZC_NEUTRAL_VBUS_HALF</code>         | Vbus / 2            | Classic fixed midpoint.                                                                     |
| <code>ZC_NEUTRAL_COMPUTED</code>          | (Vu+Vv+Vw)/3        | True star point — valid because the 3 phases are sampled simultaneously on dedicated cores. |
| <code>ZC_NEUTRAL_EXTERNAL</code>          | BEMF_N pin (AD5CH1) | External virtual-neutral resistor network.                                                  |

`FEATURE_HWZC_FILTER_COMP` (RC-lag compensation) is independent and composes with any model. The hardware-comparator ZC path ( `FEATURE_ADC_CMP_ZC` , and the ATA digital comparators on RA9/RA10/RB9) is **off** by default — the active path is the software divider-ADC ZC.

## 9. Protection layer

| Mechanism                                                  | Where       | Default                               | Notes                                                                                                                                                                                             |
|------------------------------------------------------------|-------------|---------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vbus OV</b>                                             | ADC ISR     | 2730 ≈ <b>28 V</b>                    | Above 6S full-charge (25.2 V), below board headroom.                                                                                                                                              |
| <b>Vbus UV</b>                                             | ADC ISR     | 500 ≈ <b>5 V</b>                      | Intentionally LOW for a bench PSU; <b>raise to ~1660–1755 (17–18 V) for a real 6S battery</b> .                                                                                                   |
| Regen brake / emergency hold                               | CL slew     | 2535 / 2660                           | Rescaled to the 0.0769/42 V divider; ordered below OV.                                                                                                                                            |
| <b>Software OC</b><br>( <code>FEATURE_SW_PHASE_OC</code> ) | ADC ISR     | 1500 counts                           | Trips on <b>all 3 phases (Iu/Iv/Iw) + DC bus</b> , per-signal bias auto-zeroed while disarmed.<br><code>SW_PHASE_OC_TRIP_COUNTS</code> / <code>SW_BUS_OC_TRIP_COUNTS</code> — <b>bench-tune</b> . |
| <b>ATA ILIM</b>                                            | ATA HW      | enabled,<br><code>ILIM_DAC=120</code> | Cycle-by-cycle chop (masked from nIRQ); caps inrush/stall current without faulting. <b>Tune <code>ILIM_DAC</code></b> .                                                                           |
| <b>ATA SCPCR</b>                                           | ATA HW      | 2000 mV / 7.5 μs                      | VDS short-circuit latch (gross shorts only).                                                                                                                                                      |
| <b>ATA nIRQ poll</b>                                       | main loop   | on                                    | RC6 active-low; a fault assertion → <code>ESC_FAULT</code> + <code>g_ataDiag</code> snapshot.                                                                                                     |
| <b>GDU-ready gate</b>                                      | ARM → ALIGN | on                                    | <code>g_ataReady</code> (from EnterGduNormal) blocks driving if the ATA didn't come up.                                                                                                           |

`FEATURE_HW_OVERCURRENT=0` (the MCLV dsPIC OA3/CMP3 bus-OC path does not exist here — bus OC is the software + ATA path above).

## 10. Throttle / arming

---

### Throttle sources

- **Boot default = pot** on test point TP5 (RA6/AD3AN2, `FEATURE_ADC_P0T=1`) — turn the knob for throttle.
- **GUI / GSP** (debug UART): the host can drive throttle without the pot — send `SET_THROTTLE_SRC(GSP)` then `SET_THROTTLE` (0–2000). `SET_THROTTLE_SRC(ADC)` hands control back to the knob. (Set `FEATURE_ADC_P0T=0` to make GSP the boot default instead.)
- RX/DShot compiled in but not the boot source.

### Arming (any source can arm; all gated the same way)

- **Hardware ARM toggle switch** on RD4 (**J1, TELE\_RX**) — `FEATURE_ARM_SWITCH=1`, active-low (switch → GND closed = armed, internal pull-up). Edge-triggered + debounced: **closing arms, opening kills** the bridge. It acts only on its own transitions, so it composes with the GUI (an unfitted/open switch never interferes) and a switch left closed at power-up does **not** auto-arm.
- **GUI / GSP**: `START_MOTOR` arms, `STOP_MOTOR` disarms — independent of the switch.
- Legacy momentary Start/Stop on RD5 (DShot pin) still compiled.
- Every arm path enters **ESC\_ARMED**, which waits for **throttle = 0** for `ARM_TIME` before **ESC\_ALIGN**, and ALIGN is gated by `g_ataReady` (ATA GDU must be up). Boot state is **disarmed, zero throttle**.
- RX auto-arm only fires when an RX source is explicitly selected (so a stray RX link can't arm during pot/GSP bring-up).

### GUI control without the pot — quick sequence

1. `SET_THROTTLE_SRC(GSP)` — take throttle authority from the knob.
2. `START_MOTOR` — arm (→ ALIGN once throttle 0 + GDU ready).
3. `SET_THROTTLE(n)` — ramp 0...2000.
4. `STOP_MOTOR` (or throttle 0) to stop; `SET_THROTTLE_SRC(ADC)` to return the knob.

---

## 11. Active motor parameters (⚠ bench values)

---

`FEATURE_GSP=1` → the **runtime** reads `gsp_params.c` `profileDefaults[]` (+ EEPROM overlay), **not** the `garuda_config.h` `#define`s. Active profile = `MOTOR_PROFILE=2` (slot `GSP_PROFILE_5010`), which currently carries **2810 1350KV bench-motor** data on a 24 V MCLV rig:

- polePairs 7,  $\lambda$ /Ke 583, alignDuty 3 %, rampDuty 8 %, rampTarget 3000 eRPM, cIdleDuty 4 %, maxClosedLoopErpm 260000.

**These are NOT safe defaults for an unknown 6S drone motor.** Define a real motor profile (pole pairs, align/ramp currents,  $\lambda$ ) and re-derive before powering an unknown motor on a battery. The defaults-CRC in EEPROM rejects stale overlays when the active profile's defaults change, so edited values take effect on first boot.

## 12. Caveats & known issues (carry into the bring-up)

**Must fix / decide before a real 6S battery spin**

- Motor profile** — profile 2 is the 2810 bench motor; set the real drone motor's params (§11).
- Vbus UV** — raise from ~5 V to ~17–18 V for battery use (§9).
- Current-sense calibration** —  $I_u/I_v$  (dsPIC OA gain) and  $I_w/I_{bus}$  (ATA gain  $\approx 16$ , 2 m $\Omega$ ) read on different counts/amp; the OC thresholds ( `SW_PHASE_OC_TRIP_COUNTS` , `SW_BUS_OC_TRIP_COUNTS` ) and the legacy amp-scale constants ( `OC_SHUNT_MOHM` , gain) are placeholders — calibrate against a known current. Consider splitting the phase OC threshold (dsPIC vs ATA channels).
- ATA ILIM / SCPCR** — `ILIM_DAC` , `SCTHSEL` , `SCFLT` are carried over from the reference's different shunt/FET/voltage; re-derive for 2 m $\Omega$  + BSC028N06 + 6S.

**Known limitations (not first-spin blockers)**

- No over-temperature protection** — the NTC (RA0/AD3AN5) is sampled but the value is never consumed. Add an OT trip before flight.
- AD3CH3 (Iw) conversion timing** —  $I_w$  is the 4th channel on core AD3, read ~2  $\mu$ s into the ISR with no explicit conversion-complete guard, and no ADC common clock is configured (reset defaults). Verify on the bench (scope/trace) that  $I_w$  reads a settled value; harden with an ADRDY check or move to its own core if needed.
- GSP snapshot  $I_{busAvg}$**  — the wire struct dropped  `$I_{busAvg}$`  (248  $\rightarrow$  246 B) while the host decoders expect it at offset 246 of a 248 B frame.  `$I_{busRaw}$`  /  `$I_{busMax}$`  ARE sent now (real bus current). Either restore  `$I_{busAvg}$`  (we have real bus current to populate it) or update the host decoders — coordinate the protocol version.
- nIRQ handling** — currently any nIRQ latches `ESC_FAULT` . `ReadDiag` reads the SIR registers but does not write-back-to-clear; classify + clear + debounce before relying on it at 6S (regen VDS blips could nuisance-trip).
- EEPROM is fully live** — relies on the defaults-CRC. During bring-up consider `FEATURE_PARAMS_FORCE_DEFAULTS=1` until the profile stabilizes.
- Profile-0 dead-time** ( `DEADTIME_NS=750`  $\rightarrow$  13.5 % MIN\_DUTY) — only matters if you spin profile 0; the active profile 2 (300 ns / 5.4 %) is fine.

**Cosmetic** — `foc/` (dormant) and `scope/` still have "24 kHz" comments (active hot-path files were corrected to 45 kHz).

## 13. Key feature flags (garuda\_config.h)

| Flag                                  | Value                        | Meaning                                                  |
|---------------------------------------|------------------------------|----------------------------------------------------------|
| <code>FEATURE_BEMF_CLOSED_LOOP</code> | 1                            | software ZC 6-step                                       |
| <code>FEATURE_ADC_CMP_ZC</code>       | 0                            | HW-comparator ZC off                                     |
| <code>FEATURE_HW_OVERCURRENT</code>   | 0                            | no dsPIC OA3/CMP3 bus-OC path on this board              |
| <code>FEATURE_SW_PHASE_OC</code>      | 1                            | software 3-phase + bus OC                                |
| <code>FEATURE_ADC_POT</code>          | 1                            | pot throttle on TP5/RA6 (set 0 for GSP-default)          |
| <code>FEATURE_ARM_SWITCH</code>       | 1                            | hardware ARM toggle on RD4/J1 (set 0 if not wired)       |
| <code>FEATURE_GSP</code>              | 1                            | runtime params from gsp_params + EEPROM                  |
| <code>ZC_NEUTRAL_MODEL</code>         | <code>ZC_NEUTRAL_DUTY</code> | ZC neutral model selector                                |
| <code>FEATURE_HWZC_FILTER_COMP</code> | 1                            | RC-lag compensation                                      |
| <code>FEATURE_FOC*</code>             | 0                            | FOC dormant                                              |
| <code>MOTOR_PROFILE</code>            | 2                            | active profile (2810 bench data — change for real motor) |